Developing LoRaWAN Devices Technical Recommendation TR007-1.1

NOTICE OF USE AND DISCLOSURE

LoRa Alliance®
39221 Paseo Padre Pkwy, Suite J
Fremont, CA 94538
United States

Developing LoRaWAN Devices
Technical Recommendation TR007-1.1

Authored by the LoRa Alliance Technical Committee

Technical Committee Chair and Vice-Chair:
O.SELLER (Semtech) / J.CATALANO (Kerlink)

Editor:
J.CATALANO (Kerlink)

Contributors:
S.BALL (Senet), J.CATALANO (Kerlink), J.COUPIGNY (ST), R.CRAGIE (LoRa Alliance), R.FULLER (Semtech), S.JILLINGS (Semtech), D.KJENDAL (Senet), J.KNAPP (Semtech), S.LEE (Semtech), M.LOOTSMA (TWTG), F.LU (Semtech), M.LUIS (Semtech), S.NAIK (Semtech), B.PARATTE (Semtech), H.RITTER (Minol), O.SELLER (Semtech), S.SHEN (Semtech), J.STOKKING (The Things Network), B.VERSTEEG (DISH), A.YEGIN (Actility), X.YU (Alibaba Group),

Version: 1.1
Date: September 2025
Status: Final

Contents

1 Conventions

The keywords "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP14 [RFC2119] [RFC8174] when, and only when, they appear in all capitals, as shown here.

The tables in this document are normative. The figures and notes in this document are informative.

Commands are written ***NewChannelReq***, bits and bit fields are written `CFList`, constants are written ADR_ACK_LIMIT, variables are written *N*.

1.1 Specification References

The following table describes references to specifications used throughout this document and their meaning.

Text	Meaning
LoRaWAN specifications	All of the LoRaWAN specifications in general.
LoRaWAN Link Layer specification	The LoRaWAN Link Layer Specification in general.
LoRaWAN 1.1+	Any version of the LoRaWAN Link Layer Specification that is version 1.1 or later.
LoRaWAN prior to 1.1	Any version of the LoRaWAN Link Layer Specification that is earlier than version 1.1.
LoRaWAN 1.0.4+	Any version of the LoRaWAN Link Layer Specification that is version 1.0.4 or later.
LoRaWAN prior to 1.0.4	Any version of the LoRaWAN Link Layer Specification that is earlier than version 1.0.4.
LoRaWAN 1.0.x	Any version of the 1.0 LoRaWAN Link Layer Specification (e.g., 1.0.3, 1.0.4 etc.).
[TS001-1.0.4]	An explicit reference to LoRaWAN Link Layer Specification v1.0.4, as specified in TR007 section 7.1.
[TS001-1.1]	An explicit reference to LoRaWAN Link Layer Specification v1.1, as specified in TR007 section 7.1.

2 Introduction

This technical recommendation is intended to provide guidance to end-device and LoRaWAN protocol stack developers to help ensure they produce well-behaved and interoperable products. The document is structured in a series of numbered sections. While this document does not constitute a new technical specification, normative language is used in each section so that behavior compliant with the recommendation is clear. Further, this will allow the implementor to claim compliance with a specific section of the recommendation.

3 Provisioning

LoRaWAN end-devices, both Over-the-Air-Activation (OTAA) and Activation-By-Personalization (ABP), need to be properly provisioned for operation on a LoRaWAN compliant network. OTAA devices are provisioned with a `DevEUI`, `JoinEUI`, and one or more root keys. ABP devices are provisioned with a `DevEUI`, `DevAddr`, and one-time session keys.

All end-devices are identified through the `DevEUI` (device-EUI). All end-devices are also provisioned with a shared secret (or secrets), which are used to authenticate and encrypt frames to and from the network.

3.1 Extended-Unique-Identifier (EUI)

LoRaWAN uses EUIs to identify various elements of the network architecture. End-devices are identified by a `DevEUI` and Join Servers (JS) are identified by a `JoinEUI`. The EUIs in LoRaWAN specifications are always EUI-64 (64-bit EUIs). Other architectural elements may use EUIs as identifiers in the future.

An EUI is a globally unique identifier, allocated from a pool of EUIs owned by the organization that has been allocated them by the Institute of Electrical and Electronics Engineers (IEEE). The Organizationally-Unique-Identifier (OUI) forms the most significant 24 bits of the EUI for MAC Address Block Large (MA-L) address blocks. MAC Address Block Medium (MA-M) and MAC Address Block Small (MA-S) address blocks are based on a 36-bit OUI-36. The IEEE is the Registration Authority that assigns the OUI or OUI-36 and the relevant MA-L, MA-M, or MA-S address blocks to an organization. EUIs SHALL be globally unique.

Vendors purchase OUIs from the IEEE and then allocate their `DevEUI` and `JoinEUI` addresses from the range of addresses that the OUI defines. Vendors can choose whether to keep their company name and address confidential (for an added, annual fee).

The consequence of a vendor incorrectly allocating `DevEUIs` or `JoinEUIs` (either by not owning an OUI block or improperly allocating it), is that they may collide with EUIs created by the true owner of the OUI block. This can lead to one or both devices failing to properly operate on LoRaWAN networks (this is true for both public and private networks). For this reason, implementors SHALL refrain from using OUIs that they are not entitled to (including random OUIs or other vendors' OUIs), when creating `DevEUIs` and `JoinEUIs`.

The `DevEUI` assigned to a LoRaWAN end-device SHALL be immutable, i.e., it should not be possible to change the `DevEUI` of an end-device.

3.2 Device Address (`DevAddr`)

The `DevAddr` is a short-form address used to aid in the identification of an end-device. It is allocated by the network operator and consists of NWK-ID (network ID derived from NetID) and Host-ID (in the same manner as IP addresses). The `DevAddr` is 32-bits wide and is not guaranteed to be unique. The NetID is allocated by the LoRa Alliance to member companies in good standing that request one. If the operator does not have a NetID allocated by the LoRa Alliance, they SHALL use NetID 0 or 1, the private NetIDs.

If the network operator does not follow this guideline and uses another network operator's NetID when allocating `DevAddrs`, there are two consequences. First, it is impossible to efficiently filter traffic at the gateway, creating additional backhaul usage and requiring filtering at the Network Server itself. Second, uplink frames of the misidentified device received by another network operator will not be forwarded to the real home network of the device. Instead, they will be forwarded to the network of the real owner of the NetID, which prevents the device from ever being able to take advantage of roaming network coverage.

3.3 Security Keys

In the LoRaWAN Link Layer specification, all security keys are 128-bits wide. Root keys are used only by OTAA provisioning to derive session keys, and in the Join-Request and Join-Accept messages. Session keys are used to form Message Integrity Codes (MICs) and to encrypt and decrypt all messages other than the Join-Request and Join-Accept messages.

3.4 OTAA Device Provisioning

OTAA devices must join a network to gain connectivity. The join process consists of a Join-Request issued by the end-device, followed by a Join-Accept from the network (granting access and providing a `DevAddr` and session key derivation material), and finally confirmed by the first uplink from the end-device using the new session keys. This section will describe what must be provisioned on the end-device and network to support OTAA operation.

3.4.1 Device EUI (`DevEUI`)

The `DevEUI` is a globally unique EUI-64, legitimately allocated to the end-device by the organization owning the OUI. The `DevEUI` SHALL be allocated according to the rules defined in Section 3.1.

3.4.2 Join Server EUI (`JoinEUI`)

The `JoinEUI` (also known as `AppEUI` in earlier versions of the specifications), is the EUI-64 address of the Join Server that has been provisioned with the root key material paired with the end-device. The Join Server SHALL support (at minimum), the configuration of `DevEUI/JoinEUI` pairs with the device's associated root key(s). The Join Server is responsible for validating the end-device's Join-Request, including the cryptographic material (`DevNonce`, `JoinNonce`). The `JoinEUI` is used by the end-device to address the Join-Request message.

The `JoinEUI` SHALL be allocated according to the rules defined in Section 3.1.

Provisioning and storage of the `JoinEUI` in the end-device SHOULD be flexible to allow replacement of a default Join Server with a different JS. The `JoinEUI` provisioned on the end-device SHALL identify a JS that is provisioned with the root keys of the end-device if the end-device ever roams or if the Network Server and JS are separated.

3.4.3 Root Key(s)

3.4.3.1 Root Key Generation

Root keys SHALL be generated in such a fashion that there is no derivable pattern between the root keys of multiple devices, for example, not be reversibly based on the `DevEUI` or any other easily guessed scheme, or be all 0s or all 1s or any other simple pattern.

Root keys of a set of devices SHALL exhibit a high degree of entropy (randomness) to ensure that the compromise of a single device does not lead to the compromise of additional devices. It is critical that keys conform to this model, as it makes the economics of cracking individual devices infeasible for large-scale attacks.

Two techniques are often used to generate a set of root keys. The first technique is to create a large set of unique root keys, each with its own input entropy.

The second technique is to create a single unique master root key for a set of devices and then use a one-way, keyed, cryptographic hash function to derive the unique root keys for each individual device. It is preferable that a Hardware Security Module (HSM) configured with the master root key is used to securely generate derived root keys on demand. The master root key SHALL be kept secure, as compromise of this key may compromise the entire set of devices provisioned with root keys derived from it. The master root key SHALL NOT be kept on an end-device, as this would raise the value of the attack to the point where it becomes economically feasible. These techniques are fully described in the NIST SP 800-90 series of documents (see [SP 800-90C]).

3.4.3.2 Root Key Delivery and Storage

The various parties (manufacturer, distributor, integrator, operator, etc.), SHALL strictly limit access to root key material.

An attacker is likely to try to compromise the system at its weakest point. The LoRaWAN protocol describes mechanisms that use security keys to secure traffic between the various elements of the system (see [TS002]). This design has been subjected to extensive security reviews and is robust. It is assumed that the keys are securely provisioned on the device, the Join Server, and Application Server (and perhaps the Network Server). An attacker is likely to try to compromise the system at its weakest point, which is often when the keys are provisioned to the manufacturer, delivered to the operator, or stored in the operator's systems.

The keys SHALL be stored in a secure environment with a strictly limited access policy. The delivery of keys to an authorized party (for instance, when providing new devices to a customer), SHALL use secure methods and be sent to a very small distribution. It is particularly important the customer/operator store keys in a secure manner and limit their distribution. Many security breaches can be traced to allowing operations staff access to key material, so access to keys SHALL be severely restricted.

For organizations and/or Internet of Things (IoT) applications that are particularly sensitive, it should be mentioned there are a variety of solutions on the market that offer additional layers of security for LoRaWAN systems. For example, if an end-device is subject to physical threats, its keys can be protected in tamper-resistant storage via a so-called Secure Element. In the

network backend, an HSM is a dedicated piece of hardware specifically designed to manage the cryptographic key lifecycle.

3.5 ABP Devices

Due to the vastly superior provisioning simplicity, dynamic address and session key generation, and more robust security, OTAA SHOULD be used instead of ABP.

ABP devices are provisioned with the information required for network connectivity without going through the OTAA Join Procedure. As such, they are provisioned with a `DevAddr` and session keys. In addition, the LoRaWAN specifications require the end-device be provisioned with a `DevEUI` to disambiguate its identity to the network. It is RECOMMENDED that the end-device be provisioned with all OTAA elements (`DevEUI`, `JoinEUI`, and root keys). This allows the ABP device to operate as an OTAA device, if required, and provides a robust, internal mechanism for generating session keys from statically provisioned `DevNonce` and `JoinNonce` values.

3.5.1 Device Address (`DevAddr`)

End-devices are assigned a 32-bit ephemeral device address (`DevAddr`) by the network operator when they join a network. ABP devices SHALL use a `DevAddr` assigned to them by the network operator to which they will connect or SHALL use a `DevAddr` allocated from one of the private NetID ranges.

The `DevAddr` SHALL be allocated according to the rules defined in Section 3.2.

3.5.2 Session Keys

ABP devices are provisioned with the session keys that would typically be generated through the Join Process. A full set of these keys needs to be provisioned both in the end-device and in the network to allow end-device operation. ABP devices are provisioned with the session keys whereas OTAA devices generate them through the Join Process. Session keys SHOULD be generated and handled using recommendations detailed for root keys in Section 3.4.3.

3.6 Physical Provisioning

It is RECOMMENDED that the end-device `DevEUI` be marked on the outside cover and easily legible. The LoRa Alliance has published technical recommendations, [TR005], that define a method of universal machine reading of the provisioning details, which can be used to enable automated on-boarding. It is RECOMMENDED that some type of machine-readable marking of the basic provisioning information (`DevEUI` and `JoinEUI`), exist on the device exterior to facilitate rapid and reliable device deployment. Security keys SHALL NOT be printed or displayed in any way on the end-device.

4 LoRaWAN Protocol Stack

4.1 Join Procedure

4.1.1 Description

Only an end-device SHALL initiate the Join Procedure. Use of the Join-Request SHOULD only be used after commissioning a factory reset, loss of connectivity with the network, when one of the frame counters has reached its maximum value, or new session keys are generated for security reasons.

Join Requests create additional work for the Network Server, Join Server, and Application Server, as well as additional network traffic to exchange key material, etc. Limiting Join Requests will reduce unnecessary downlink and MAC command traffic.

4.1.2 Recommended Practice

If the device is powered down completely, it SHOULD store the values described in Section 4.13 for use when it powers up again. For a static device, there may be a network operator requirement to occasionally trigger a Join Request message, allowing them to regenerate their network and application session keys, but this SHOULD be limited to once a month at most.

Transmission of the Join-Request is governed by the retransmission back-off procedure specified in the LoRaWAN Link Layer specification. This includes the case when the end-device initiates the Join Procedure due to the temporary loss of power. In some end-devices, as the power system nears depletion, the end-device retains enough charge to transmit an uplink (Join-Request), but the act causes a brown-out reset of the host CPU/MCU. The end-device SHALL guarantee that the back-off duty cycle is observed even in this condition.

In LoRaWAN 1.1+, the network MAY request the end-device initiate the Join Procedure, which may be required to refresh session keys or to reallocate `DevAddr`. As LoRaWAN 1.0.x does not support this ability, the end-device SHOULD support an application-layer means of doing so.

4.2 Fixed Channel Plan Join Process Optimization

4.2.1 Description

The gateway in range of the end-device may be operating on 64 or more channels or on fewer channels due to limitations of the deployed gateway radios. Optimizing the Join process will typically allow the end-device to connect in fewer attempts than when using a purely random or purely sequential channel selection process.

4.2.2 Recommended Practice

In the fixed channel regions of US915 and AU915, there are eight eight-channel banks in a 64-channel gateway, each with eight 125 KHz channels plus one 500 KHz channel. Join Request attempts using US915 SHOULD be at DR0 for the 125 KHz channels and DR4 for

the 500 kHz channels. For AU915, these Join Requests SHOULD be sent at DR2 and DR6 respectively. Join Request attempts SHOULD select one of the 8 channel banks and then randomly select a channel within that channel bank. A channel from each distinct channel bank SHOULD be used without repeating bank usage. After a single channel has been attempted from each channel bank, another cycle of attempts SHOULD be made using one of the remaining seven unused channels from each bank, again in random fashion. This continues until all channels in each bank are used once in a cycle of 72 Join Request attempts.

4.3 Detecting Loss of Network Connectivity

4.3.1 Description

An end-device may lose connectivity with a network for a variety of reasons:

1. The end-device has moved and is no longer in coverage, either while using its current radio frequency (RF) configuration or regardless of its RF configuration.
2. The network conditions have changed (new interference, gateways have been reconfigured, etc.), resulting in loss of coverage.
3. The network has lost state synchronicity with the end-device (session keys were lost, frame counters are out-of-sync, etc.).
4. The network operator has ceased operating the network.

The end-device may use several techniques to detect and mitigate these conditions.

4.3.2 Recommended Practice

End-devices with Adaptive Data Rate (ADR) enabled (both the end-device and the network assert the ADR bit), SHALL follow the ADR back-off procedure described in the LoRaWAN Link Layer specification. This procedure uses `ADRAckReq` and the receipt of Class A downlinks to test connectivity and will gradually restore the end-device's default ADR configurations in a stepwise manner. Ultimately the end-device is configured to use all default channels at maximum transmit power and lowest data rate. For fixed channel plan regions, this requires that ALL channels are enabled. For dynamic plan regions, this requires that the default (or join) channels are enabled in addition to any dynamically configured channels. `ADRAckReq` remains set after returning to default ADR configuration. An additional ADR_ACK_LIMIT uplinks sent with no Class A downlink received is strongly RECOMMENDED. When no Class A downlinks have been received after that extended period, the end-device MAY consider network connectivity to be lost.

End-devices not being controlled through ADR MAY test network connectivity in a similar manner but SHALL use a technique other than `ADRAckReq` to request a downlink from the network. If these methods are not part of the end-device's normal operating mode, it is RECOMMENDED that they be used only occasionally (in the same manner `ADRAckReq` is used). End-devices that employ these methods as part of their normal operation, but are also controlled through ADR, MAY use these techniques to augment the `ADRAckReq` based back-off. Several examples of such techniques are:

1. Send a confirmed uplink (which may contain no application payload), to request an acknowledgement from the network.

2 Send a ***LinkCheckReq***, which requests an ***LinkCheckAns*** from the network.
3 Send an application-layer message, which requests an application-layer response in a Class A window.

It is further RECOMMENDED that end-devices only deploy these techniques in a manner that mirrors the ADR back-off algorithm:

1 The end-device reverts to default ADR and channel configuration in a stepwise manner.
2 Any Class A downlink halts the back-off procedure and confirms network connectivity.
3 The network is given multiple attempts to send the confirming Class A downlink during each back-off step.
4 Loss of connectivity is only determined after multiple attempts at default RF configuration – keep in mind that in many fixed channel plan regions, only 1/8th of transmitted messages may be received by some gateways – the determination of loss of connectivity MUST take this into account and not prematurely assume connectivity is lost.

4.4 Reacting to Loss of Network Connectivity

4.4.1 Description

If an end-device determines it is no longer in contact with the network (using the procedures described above), it MAY take additional steps to mitigate connectivity issues not related to RF coverage (which are addressed above). These root causes are:

1 Loss of state synchronicity between the end-device and the network (session keys or frame counters).
2 Cessation of operation or contractual agreement between the end-device owner and the network operator and the need to establish connectivity with a new operator.

ABP end-devices have no recourse in these situations, therefore the recommendations are scoped to OTAA end-devices.

4.4.2 Recommended Practice

End-devices that support LoRaWAN 1.1+ SHALL use ReJoin Type 1 messages to probe the network for an opportunity to synchronize end-device security and radio state (`DevAddr`, session keys, frame counters, channel plans, and downlink RF parameters). This technique mitigates both cases described above without impacting the end-device's ability to continue to deliver uplink messages.

End-devices that support versions of LoRaWAN prior to 1.1 MAY use Join-Request messages to attempt to restore connectivity in these cases. While this technique mitigates both cases described above, it does so at the expense of the end-device's ability to continue to deliver uplink messages. Once the end-device issues a Join-Request, uplink messages are no longer valid until a valid Join-Accept is received and processed. In the case of downlink-only connectivity issues, the act of sending the Join-Request will cause the end-device to no longer be able to transmit any uplink messages until a valid Join-Accept is received. As a result, the decision to send a Join-Request while a security session is already established SHOULD only

be used as a last resort. As fully specified in the LoRaWAN Link Layer specification, end-devices SHALL comply with the retransmission back-off behavior during an attempt to re-establish connectivity with the network through the Join Procedure.

4.5 Rolling Session Keys

4.5.1 Description

The end-device, the network, or the application may determine that the session keys for the end-device's current security session need to be refreshed. This may be due to suspected security compromises, the lifetime of the existing session, frame counters are approaching maximum values, or other security considerations.

The ability to re-key the end-device is limited to OTAA end-devices.

4.5.2 Recommended Practice

For end-devices that support LoRaWAN 1.1+, the end-device SHALL initiate a re-keying event through the transmission of a ReJoin Type 2 message. Network-side re-keying SHALL be initiated via the ***ForceReJoinReq*** MAC command. Application re-keying MAY be initiated via application-layer messaging or through an interface to the Network Server requesting it to send the ***ForceReJoinReq*** MAC command. This enables the graceful re-keying of the-end-device without interrupting the delivery of uplink messages.

End-devices that support versions of LoRaWAN prior to 1.1 MAY use Join-Request messages to roll session keys. This technique re-keys the device at the expense of the end-device's ability to continue to transmit uplink messages. Once the end-device issues a Join-Request, uplink messages SHALL NOT be transmitted until a valid Join-Accept is received and processed. In the case of downlink-only connectivity issues, the act of sending the Join-Request will cause the end-device to no longer be able to deliver any uplink messages. As a result, the decision to send a Join-Request while a security session is already established SHOULD only be used as a last resort. There is no network-side facility to initiate a re-keying event. It is RECOMMENDED that application layer re-keying initiation be supported by LoRaWAN 1.0.x end-devices.

4.6 DevNonce Values SHALL Not Be Reused

4.6.1 Description

LoRaWAN requires `DevNonce` values (for any given `DevEUI/JoinEUI` pair), never be reused. The simplest and most effective way to accomplish this is for the end-device to increment the `DevNonce` for each Join-Request transmitted, starting at zero for the first Join-Request. Refer to [TS001] for more information about `DevNonce`.

Join Servers keep a limited history of `DevNonce` values previously used by an end-device to detect a Join-Request replay and will reject a Join-Request if it contains a `DevNonce` that has been previously used. Using an incrementing `DevNonce` enables the Join Server to practically

detect all Join-Request replays instead of only being able to detect replays of a limited number of random `DevNonces`.

LoRaWAN 1.0.4+ and 1.1+ make this behavior mandatory.

4.6.2 Recommended Practice

For LoRaWAN prior to 1.0.4, refer to [TR001-1.0.0].

4.7 Avoiding Synchronous Behavior

4.7.1 Description

To maximize network capacity and message receipt success, it is important that end-devices implement randomness in both the transmission timing and channel selection. Randomness in these two dimensions helps prevent collisions and uplink spikes, which can artificially limit gateway and network capacity.

4.7.2 Recommended Practice

End-devices SHALL create a pseudo-randomly sorted list of enabled channels and iterate through this list with each transmission. The list SHALL be re-sorted when new channels are enabled or disabled. The end-devices SHALL ensure that the pseudo-random number generators used to create this list are seeded with truly random values to prevent multiple end-devices from systematically using identically sorted lists. Some regulatory regions require every channel to be used equally, and this technique also meets that requirement.

End-devices SHALL add a pseudo-random delay to all periodic transmissions (including Join Requests and retransmissions controlled by `NbTrans` or the application-layer). The end-device SHALL ensure that the pseudo-random number generators used to create this list are seeded with truly properly random values (see [RFC4086] for examples and methodology), to prevent multiple end-devices from systematically using the same delay values.

Groups of end-devices SHALL NOT be coordinated to transmit at the same real-time. Even loose coordination of transmit time places undue burden on the network. An example of behaviors to be avoided are:

1. Daily uplinks at midnight of a day's worth of sensor readings.
2. Weekly validation of the configuration of a fleet of devices to occur with a 12-hour window every week.
3. Transmitting subsequent confirmed uplinks, having not received an acknowledgement at exactly 2500 mSec after the previous uplink.

4.8 Avoiding Congestion Collapse

4.8.1 Description

A network congestion collapse occurs when an event triggers widespread and repeated communication attempts. For example, many end-devices use a reed-switch for basic, non-

contact control (power on/off, re-Join, etc.). Should an earthquake occur near a large population of end-devices, it is probable that they will all simultaneously reboot and initiate the Join Procedure. The fact that these messages are all sent simultaneously could overwhelm the gateway front ends, the gateway backhauls, or even the Network Server or Join Server. In this case, no response would be sent, causing the end-devices to again send the Join-Request. In the worst case, the end-devices react by sending the requests at an even faster pace to try to quickly receive a response. When large groups of end-devices behave this way, it can lead to a condition in which the network cannot service any of them.

There are many such trigger events that can impact any end-device and any network, some first order (like an earthquake or power outage), others second or third order (a chain of conditions that ultimately lead to the collapse). As a result, mitigation techniques need to be universally implemented.

Any time the end-device sends an uplink that requires a response (ACK, Join-Accept, MAC Command response, Application-Layer response, etc.), an opportunity for congestion collapse is introduced. If the end-device reacts to the lack of response by sending another uplink that again requires a response, then it SHALL implement techniques to avoid instigating a congestion collapse.

Even end-device communications that do not demand a response can lead to temporary collapse if the end-device enters a mode where it dramatically increases the pace at which it sends uplinks for an extended period of time.

4.8.2 Recommended Practice

Whenever possible, end-devices SHOULD use communication techniques that do not require responses. In addition to avoiding congestion collapse, this technique also preserves the downlink capacity of the network, which for LoRaWAN networks is less than the uplink capacity.

End-devices SHALL use the randomization techniques of both transmit time and channel described above to avoid synchronous transmissions among groups of end-devices. This will help mitigate the temporary congestion possible even when responses are not requested.

In response to a missed response, the end-device SHALL decrease the pace of transmission (increase the periodicity of transmission). A reasonable back-off strategy described in [TS001-1.0.4] is reproduced here:

> *Uplink frames that:*
>
> - *require an **acknowledgment or answer** from the Network or an Application Server and are **retransmitted** by the end-device if the acknowledgment or answer is not received*
>
> ***and***
>
> - *can be triggered by an **external** event causing **synchronization** across a large (>100) number of end-devices (power outage, radio jamming, network outage, earthquake…)*

can trigger a catastrophic, self-persisting, radio network overload situation.

***Note:** A typical example of such an uplink frame is a Join-Request, if the implementation of a group of end-devices decides to reset the MAC layer, in the case of a network outage. The entire group of end-devices will start broadcasting Join-Request uplinks and will stop only upon receiving a Join-Accept from the network.*

For those frame retransmissions, the interval between the end of the RX2 slot and the next uplink retransmission SHALL be random and follow a different sequence for every end-device (for example using a pseudo-random generator seeded with the end-device's address). The transmission duty cycle of such a frame SHALL respect local regulations and the following limits, whichever is more constraining:

Aggregated during the first hour following power-up or reset	$T_0 < t < T_{0+1}$	*Transmit time* *< 36 s per hour*	*1% duty cycle*
Aggregated during the next 10 hours	$T_{0+1} < t < T_{0+11}$	*Transmit time* *< 36 s per 10 h*	*0.1% duty cycle*
After the first 11 hours, aggregated over 24 hours, where N refers to days, starting at 0	$T_{0+11} + N \times (24 \text{ hours/day}) < t < T_0 + 35 + N \times (24 \text{ hours/day}), N \geq 0$	*Transmit time* *< 8.7 s per 24 h*	*0.01% duty cycle*

Additional back-off algorithms MAY be implemented within each period described above.

4.9 Discontinuing Retransmissions

4.9.1 Description

End-devices will sometimes issue retransmissions (uplink messages with the same content and frame counter as the previous message), to improve message delivery. This is controlled via the `NbTrans` parameter. Any reception by the end-device in a Class A window explicitly indicates that the message has been received by the network and no additional retransmissions of the same frame counter are needed.

4.9.2 Recommended Practice

When an end-device retransmits an uplink message multiple times because of `NbTrans` > 1, it SHALL stop retransmitting the message as soon as it has received any valid Class A downlink. This behavior is fully specified and REQUIRED by LoRaWAN 1.0.4+.

4.10 Default Channels

4.10.1 Description

There are two styles of regional channel plans defined in [RP002]: dynamic and fixed. Default channels are the channels that are required to be implemented on the end-device for each region. These channels MAY be disabled by the network, but all end-devices begin operation using the default channels and re-enable all default channels as the last step of ADR back-off.

Dynamic channel plan regions (EU868, CN779, EU433, IN865, KR920, AS923-1, AS923-2, AS923-3 as of [RP002]) define a small number of default channels (two or three depending on region). Additional channels are dynamically created by the network via the `CFList` or ***NewChannelReq*** commands.

For fixed channel plan regions (US915, AU915, CN470 as of [RP002]), a large set of default, fixed channels are defined. Channels are enabled and disabled via the `CFList` or ***LinkADRReq*** commands. In a future version of the LoRaWAN Layer 2 specification, fixed channel plan regions will also support the ***NewChannelReq*** command for defining new channels.

Networks MAY be deployed with gateways that support a variety of different channels, both in terms of number of channels and frequencies supported. To be able to use these various network configurations, end-devices need to correctly implement default channel behaviors.

4.10.2 Recommended Practice

End-devices SHALL begin operation with all default channels enabled. OTAA devices will use all these channels for the Join Procedure. As the last step of ADR back-off, the end-device SHALL enable all default channels.

End-devices SHALL support the full range of channel indexes defined in [RP002].

These behaviors are clearly specified by LoRaWAN 1.0.4+.

4.11 FcntUp/FcntDown SHALL Use 32 Bits

4.11.1 Description

LoRaWAN prior to 1.0.4 allows for the use of 16-bit or 32-bit wide frame counters. To maximize end-device security and reduce the risk of a message playback attacks, all implementations SHALL use 32-bit wide counters.

4.11.2 Recommended Practice

End-devices SHALL implement 32-bit unsigned integer counters for all frame counters.

LoRaWAN 1.0.4+ mandates 32-bit wide frame counters.

4.12 Frame Counters Incremented Only With Frame Transmission

4.12.1 Description

The frame counter is used for both security (for MIC generation/validation and encryption/decryption), and detection of lost frames. By only incrementing the frame counter after transmission, end-devices, Application Servers, and Network Servers can reliably use the frame counter to detect lost frames. In turn, this information could influence RF configuration to optimize communication reliability and efficiency.

4.12.2 Recommended Practice

End-devices, Application Servers, and Network Servers SHALL only increment (by one) frame counters after a message transmission has been attempted.

4.13 OTAA REQUIRED Persistent Values

4.13.1 Description

End-devices operating in OTAA mode are REQUIRED to persist several values to remain compliant with the LoRaWAN Link Layer specification.

4.13.2 Recommended Practice

For end-devices that will initiate a Join Procedure following a reset or loss of power, the following values SHALL be persisted. End-devices that will NOT initiate a Join Procedure following a reset or loss of power SHALL persist these values as well as those described for ABP end-devices:

- **`DevEUI`**
- **`JoinEUI`**
- **Root keys (AppKey/NwkKey)**
- **`DevNonce`**
  The `DevNonce` SHALL be implemented as a counter, the last value used SHALL be persisted to ensure no previous values are reused.
- **`JoinNonce`**
  The `JoinNonce` needs to be validated against reuse by the end-device. If the JS supports incrementing `JoinNonce` values, as recommended in [TR001-1.0.0], a last value SHALL be persisted for validation. If the JS does not support incrementing `JoinNonce` values, the end-device SHOULD persist a reasonable number of most recently seen values.

In LoRaWAN 1.0.4+, `DevNonce` SHALL be implemented as a counter.

4.14 ABP REQUIRED Persistent Values

4.14.1 Description

End-devices operating in ABP mode are REQUIRED to persist several values to remain compliant with the LoRaWAN Link Layer specification and ensure that they remain in contact with the network.

4.14.2 Recommended Practice

All ABP end-devices SHALL persist the following values through a reset or loss of power:

- **`DevEUI`**
- **`DevAddr`**
- **SessionKeys**
- **Frame counters** (both uplink and downlink)

End-devices that do not support ***ResetInd*** SHALL persist the additional values through a reset or loss of power:

- **Channel List** (frequencies and enabled channels)
- **Data rate**
- **TxPower**
- **NbTrans**
- **MaxDutyCycle**
- **RX2Frequency**
- **RX1DROffset**
- **RX2DataRate**
- **RXTimingDelay**
- **MaxEIRP**
- **DownlinkDwellTime**
- **UplinkDwellTime**

If an end-device is operating with Class B enabled, the following additional values SHALL be persisted through a reset or loss of power:

- **PingSlotPeriodicity**
- **BeaconFrequency**
- **PingSlotFrequency**
- **PingSlotDataRate**

4.15 Adaptive Data Rate Support

4.15.1 Description

ADR is used by the Network Server to optimize the configuration of the end-device to:

1. Maximize connectivity
2. Minimize airtime

667 3 Minimize end-device power consumption

668 ADR controls the channel plan, data rate, transmit power, and number of retransmissions used
669 by the end-device. These attributes are dynamically adapted as the network observes
670 changes in the RF conditions and gateway configurations supporting the end-device.

671 In all cases, the network is in the best position to determine the optimum ADR configurations
672 for the end-device as it enjoys a network-wide view, where the device has no visibility outside
673 of its own behavior. This is true even for mobile devices.

674 ### 4.15.2 Recommended Practice

675 ADR SHOULD be fully supported and enabled by default. The end-device owner SHOULD
676 coordinate with the network operator to define a device management schema (i.e., profile) that
677 optimizes its connectivity. This is particularly true for end-devices with special considerations
678 (mobile, nomadic, or other widely varying RF conditions).

679 ## 4.16 ADR Back-Off

680 ### 4.16.1 Description

681 The LoRaWAN Link Layer specification describes the mechanism by which ADR-controlled
682 end-devices restore connectivity by executing a stepwise algorithm that restores maximum
683 transmit power, gradually decreases data rate, and ultimately re-enables all default channels.
684 The pace of this back-off is controlled by a pair of parameters: ADR_ACK_LIMIT and
685 ADR_ACK_DELAY. By default, ADR_ACK_LIMIT is 64 uplink messages without a Class A
686 downlink and ADR_ACK_DELAY is 32 uplink messages without a Class A downlink.

687 ### 4.16.2 Recommended Practice

688 End-devices SHOULD implement ADR and the ADR back-off algorithm with the default values
689 of ADR_ACK_LIMIT and ADR_ACK_DELAY, defined in [RP002]. If the end-device
690 implements values other than the default for ADR_ACK_LIMIT and ADR_ACK_DELAY, the
691 end-device SHALL communicate these values to the network operator out-of-band, during the
692 provisioning process.

693 ## 4.17 Duty Cycle Limitations

694 ### 4.17.1 Description

695 The LoRaWAN Link Layer specification defines some duty cycle limitations and controls over
696 the end-device to improve network optimization, however, regulatory regions around the world
697 may enforce specific duty cycle limitations in addition to those specified by LoRaWAN. These
698 limitations may be different by frequency band, channel bandwidth, or transmit power. Duty
699 cycle limitations may apply to the gateway as well as the end-device, which may impact the
700 network's ability to send downlinks in response to requests from the end-device.

4.17.2 Recommended Practice

The end-device SHALL respect the lower of the two duty cycle limitations (LoRaWAN or Regulatory). Most often, a LoRaWAN network will not set a duty cycle limitation, and the end-device SHALL respect the regulatory duty cycle limitations.

4.18 Transmit Power

4.18.1 Description

To efficiently support the greatest variety of operating conditions, it is useful for the end-device to implement transmit power control across the widest reasonable range of power levels, which are specified in [RP002].

Different regulatory regions support a variety of maximum transmit power levels. These may be specified as antenna port conducted power, EIRP, power-spectral-density, or other measured values.

4.18.2 Recommended Practice

End-devices SHOULD support the highest practical transmit power in the region they are certified to operate in.

End-devices SHOULD support the full range of power control specified in [RP002].

End-device manufacturers designing devices targeted (or which may be targeted in future), for multiple regulatory regions SHOULD design them to support the maximum practical transmit power across those regions.

In LoRaWAN 1.0.4+, a minimum power control range is mandatory. An end-device SHALL support a minimum power control range of 14dB, or support a minimum transmit power of 2dBm.

4.19 Class B/C Cell Update

4.19.1 Description

End-devices operating in Class B or Class C mode MAY need to occasionally inform the network about which gateway (cell) to use for downlinks. While Class A uplinks continuously inform the network as to which gateways are best used for downlinks, end-devices receive Class B/C downlinks without an immediately preceding uplink. As the RF conditions or network change, or if the end-device moves, it is necessary for the end-device to send an uplink to inform the network that a new set of gateways may need to be selected for downlinks, however, it is best to minimize the number of uplinks used solely for this purpose.

If the network is not informed of the end-device's new cell location, Class B and Class C downlinks may be transmitted by gateways that are not within range of the end-device.

4.19.2 Recommended Practice

End-devices MAY use Class B beacons (regardless of the end-device Class), to detect changes in the set of gateways connecting it to the network. Class B beacons MAY contain gateway-specific fields, including GPS coordinates of the gateway, a NetID, and GatewayID or other proprietary extensions. If this technique is used to detect cell changes, the end-device SHOULD keep a list of recent gateway identifiers. This list SHOULD be sized to track a reasonable number of local gateways. If a new gateway identifier is added to the list, the end-device SHOULD send an uplink frame to allow the network to update its cell location. The end-device SHOULD age entries out of this list if no beacon containing the entry's gateway ID has been received for a long period of time.

For end-devices operating in areas where beacons (or beacons with gateway-specific fields identifying the gateway), are not available, cell change detection is not possible. If the end-device includes an accelerometer, GPS/GNSS or other means to detect that it has moved to a new location, the end-device MAY elect to transmit an uplink frame to inform the network of a possible cell change. The end-device SHOULD send such a frame only if it detects that a sufficiently large change in location may have occurred.

All end-devices operating in Class B or Class C mode SHOULD send periodic uplink messages to keep the network informed of its cell location. The periodicity of these messages MAY be very low if the above event triggered messages are also implemented.

4.20 Maintaining Time Synchronization

4.20.1 Description

End-devices MAY require that they maintain time synchronization with the network (all Class B-enabled devices SHALL maintain time synchronization with the network). LoRaWAN defines the Class B Beacon as a mechanism by which an end-device MAY maintain such time synchronization. A Class B Beacon, like most beacons in general, is not cryptographically protected (it is protected only from incidental RF corruption via CRC fields). This is because any form of cryptographic protection requires the use of a credential to verify the beacon, and distribution of this credential prior to discovering the beacon, in addition to the increased size of the beacon, can pose a number of challenges. As such, an attacker may send beacons with any values for the `Time` and `GwSpecific` fields they chose. In addition, an attacker may shift the beacon delivery time by jamming the original beacons and slowly shifting a replayed beacon to the end-device at a slightly different time than is required by specification.

There are several techniques the end-device and the network can use to mitigate the time-based element of these attacks, however these techniques will not mitigate against modification of the beacon `GwSpecific` field. The techniques employed by the end-device and network are application- and situation-specific. The beacon is best used as a mechanism to deliver hints to end-devices, which would require an end-device to take further action to validate that information.

4.20.2 Recommended Practice

4.20.2.1 Periodic Validation

End-devices that leverage the Class B Beacon for time synchronization SHALL periodically solicit the current network time by issuing a ***DeviceTimeReq*** and receiving a ***DeviceTimeAns***. If the received time is consistently contradictory to the time received in the beacon, the end-device knows the beacon is unreliable and another source of time synchronization is REQUIRED.

4.20.2.2 Downlink ping

Class B end-devices are able to use the reception time of valid unicast downlink ping messages to confirm the network time (as these are precisely scheduled). It remains possible for an attacker to shift these messages over time, and as result, recommendation 4.20.2.1is still REQUIRED to confirm the time remains in sync.

The unicast downlink ping message SHALL contain either an implicit or explicit value that confirms the current time to the end-device. An example of an implicit value is an application-layer MIC that is salted with the ping-slot time. An example of an explicit value is the contents of the ***DeviceTimeAns*** corresponding to the ping-slot time in the body of the message.

4.20.2.3 Multicast-group Beaconing

End-device may join a multicast group whose downlink ping messages include the current time. These messages are cryptographically protected and therefore securely deliver time to the end-devices in the group. An example of this technique is described in [TR012] Section 4.3.2.

5 General Product Security

The primary purpose of this document is to provide recommendations that pertain to the LoRaWAN-specific parts of the LoRaWAN-enabled product, however there may be considerations for overall product security that ultimately depend on the application functionality the LoRaWAN-enabled product provides to the overall system. For example, a simple LoRaWAN-enabled environmental monitoring device used in grape growing, that is one of a thousand spread across the whole vineyard, may not need the same security considerations as a LoRaWAN-enabled door lock.

Because of this diversity of applications, it is difficult to be overly prescriptive regarding security features that apply generally to all LoRaWAN-enabled products. This section highlights certain security features that may need to be considered when designing a LoRaWAN-enabled product.

5.1 External Schemes

There are a number of external schemes run by various organizations that can assist with improving overall product security. These range from guidelines to strict certification, and certain products targeted at a specific type of deployment may have additional requirements for specific security certification. This is often the case in, for example, critical applications, however certification is becoming increasingly essential for all IoT products (which includes LoRaWAN-enabled products). Therefore, it is RECOMMENDED that product developers consider external security certification for their products.

5.2 Interfaces

A LoRaWAN-enabled product may have many interfaces that allow data in and out of the device.

- Debug interfaces (e.g., JTAG, SWI), that are used for development purposes, SHALL be disabled in production devices.
- Interfaces used for testing purposes in production SHOULD be disabled for deployed devices.
- Application interfaces SHALL NOT provide any means to modify any LoRaWAN operations attributes other than configuration attributes.
- Application interfaces SHALL NOT provide any means to read any confidential LoRaWAN attributes, for example root or session keys.

5.3 Device Integrity

A LoRaWAN-enabled product MAY require additional measures to ensure device integrity. These measures may include:

- Secure boot
- Firmware update

A LoRaWAN-enabled product SHOULD implement the LoRaWAN Firmware Management Protocol Specification [TS006] to allow firmware updates over-the-air (FUOTA).

A LoRaWAN-enabled product SHOULD implement a QR code as described in the LoRaWAN® Device Identification QR Codes [TR005].

6 Glossary

ABP	Activation-By-Personalization
ADR	Adaptive Data Rate
AppEUI	(Deprecated) Join Server EUI, now called JoinEUI
AS	Application Server
DevAddr	Device Address
DevEUI	Device EUI
DR	Data rate
EUI	Extended-Unique-Identifier
FUOTA	Firmware Update Over-The-Air
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
HSM	Hardware Security Module
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
JoinEUI	Join Server EUI
JS	Join Server
MAC	Medium Access Control
MA-L	MAC Address Block Large
MA-M	MAC Address Block Medium
MA-S	MAC Address Block Small
MIC	Message Integrity Code
NS	Network Server
OTAA	Over-the-Air-Activation
OUI	Organizationally Unique Identifier
RF	Radio Frequency
RX1/RX2	Received Window
Secure Element	A secure operating system in a tamper-resistant processor chip or secure component

7 Bibliography

7.1 References

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, March 1997

[RFC4086] Schiller, J. and Crocker, S., “Randomness Requirements for Security”, RFC4086, June 2005

[RFC8174] Leiba, B., “Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words”, BCP 14, RFC8174, May 1997

[RP002]: RP002-1.0.4 LoRaWAN Regional Parameters, LoRa Alliance Technical Committee, September 2022.

[SP 800-90C] NIST SP 800-90C Recommendation for Random Bit Generator (RBG) Constructions, July 3 2024

[TR001-1.0.0]: Technical Recommendations for Preventing State Synchronization Issues around LoRaWAN® 1.0.x Join Procedure, August 2018

[TR005]: LoRaWAN Device Identification QR Codes, LoRa Alliance Technical Committee, October 2020.

[TS001-1.0.4]: LoRaWAN Link Layer Specification version 1.0.4, LoRa Alliance Technical Committee, October 2020

[TS001-1.1]: LoRaWAN Link Layer Specification version 1.1, LoRa Alliance Technical Committee, October 2017

[TS002]: LoRaWAN Backend Interfaces Specification version 1.0, LoRa Alliance Technical Committee, October 2017

[TS005]: LoRaWAN Remote Multicast Setup Specification v2.0.0, LoRa Alliance Technical Committee, April 2022

[TS006]: LoRaWAN Firmware Management Protocol Specification v1.0.0, LoRa Alliance Technical Committee, April 2022

[TS009]: LoRaWAN Certification Protocol Specification version 1.0.0, LoRa Alliance Certification Committee, January 2020.